

SOFTWARE ENGINEERING

Agile Development

Software Engineering Methodology for Modern Development

Iterative

Collaborative

Adaptive

Review

Prescriptive Process Models

Traditional Characteristics

- **Discipline**
  Structured approach with defined phases and gates
- **Systematic**
  Step-by-step methodology with clear progression
- **Rigor**
  Strict adherence to process and documentation standards
- **Work Products**
  Comprehensive documentation and deliverables

The Challenge

Changes occur everywhere in modern software development, making traditional prescriptive models struggle to keep pace.

The Problem

Rigid processes cannot adapt to rapid changes

Traditional models assume stable requirements

The Need

More adaptive, flexible approaches to software development

Requirements

Frequent changes and evolving needs

Business

Market demands and competitive pressure

People

Team changes and stakeholder dynamics

Technologies

Rapid evolution and new platforms

The Foundation

Manifesto for Agile Software Development

"We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:"

Individuals and Interactions

over processes and tools

Customer Collaboration

over contract negotiation

Working Software

over comprehensive documentation

Responding to Change

over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

— by Kent Beck et al.

Core Concept

What is "Agility"?

Effective Response to Change

Rapid and adaptive response to changing requirements, market conditions, and technological shifts.

Effective Communication

Continuous dialogue among all stakeholders ensuring alignment and shared understanding.

Customer Integration

Drawing customers onto the team as active participants throughout development.

Team Empowerment

Self-organizing teams that maintain control of the work performed.

End Result

Rapid, Incremental Delivery

Continuous delivery of working software in small, valuable increments

Key Insight

Agility is not about speed alone—it's about **adaptability, collaboration, and delivering value continuously.**

Process Characteristics

An Agile Process

Customer-Driven Requirements

Driven by **customer descriptions** of what is required (scenarios), ensuring development aligns with real user needs.

Short-Lived Plans

Recognizes that **plans are short-lived** in dynamic environments and embraces continuous planning over rigid upfront design.

Iterative Development

Develops software **iteratively** with heavy emphasis on construction activities and continuous feedback loops.

Core Philosophy

An agile process is not a rigid methodology but a **flexible, adaptive approach** that embraces change and puts people first.

Multiple Increments

Delivers working software in small, valuable increments

Adaptive Approach

Adapts as changes occur throughout development

Key Benefits

- ✓ Faster time-to-market
- ✓ Higher customer satisfaction
- ✓ Better quality software
- ✓ Improved team morale

Agile Mindset

"Responding to change over following a plan"

The 12 Principles

Agility Principles (Part 1)

Principles 1-4: Customer Focus and Collaboration

1

Customer Satisfaction Through Early Delivery

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

Priority: Customer value delivery

2

Welcome Changing Requirements

Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.

Embrace change as opportunity

3

Deliver Working Software Frequently

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Cadence: Weeks, not months

4

Business and Developers Collaboration

Business people and developers must work together daily throughout the project.

Daily collaboration is essential

Theme: These first four principles establish the foundation: customer focus, embracing change, frequent delivery, and continuous collaboration.

The 12 Principles

Agility Principles (Part 2)

Principles 5-8: People, Communication, and Progress

5 Motivated Individuals

Build projects around **motivated individuals.** Give them the environment and support they need, and trust them to get the job done.

6 Face-to-Face Conversation

The most **efficient and effective method** of conveying information to and within a development team is face-to-face conversation.

7 Working Software as Measure

Working software is the primary measure of progress. Functionality that delivers value is the ultimate indicator of success.

8 Sustainable Development

Agile processes promote **sustainable development.** The sponsors, developers, and users should be able to maintain a constant pace indefinitely.

People First

Motivated individuals and trust-based environments

Direct Communication

Face-to-face conversation over documentation

Steady Pace

Avoid burnout, maintain quality indefinitely

Theme: These principles emphasize people over process, direct communication over intermediaries, and sustainable pace over heroic effort.

The 12 Principles

Agility Principles (Part 3)

Principles 9-12: Technical Excellence and Continuous Improvement

9

Technical Excellence Enhances Agility

Continuous attention to technical excellence and good design enhances agility. Quality enables speed.

10

Simplicity is Essential

Simplicity — the art of maximizing the amount of work not done — is essential. Do only what adds value.

11

Self-Organizing Teams

The best architectures, requirements, and designs emerge from **self-organizing teams.**

12

Regular Reflection and Adjustment

At regular intervals, the team **reflects** on how to become more effective, then tunes and adjusts its behavior accordingly.

Technical Excellence

Quality and good design enable speed

Simplicity

Maximize work not done

Continuous Improvement

Regular team reflection and adaptation

Complete Framework

All 12 principles work together to create a comprehensive philosophy for software development.

The Human Element

Human Factors in Agile Teams

Emphasize "People Factors"

Agile recognizes that **people are the most important factor** in software development success. Process and tools are secondary to individuals and interactions.

Process Model Adapts to Team

The **process model adapts to the team and its members,** not the other way around. Agile frameworks are flexible and can be tailored.

Flexibility over rigidity

Key Insight

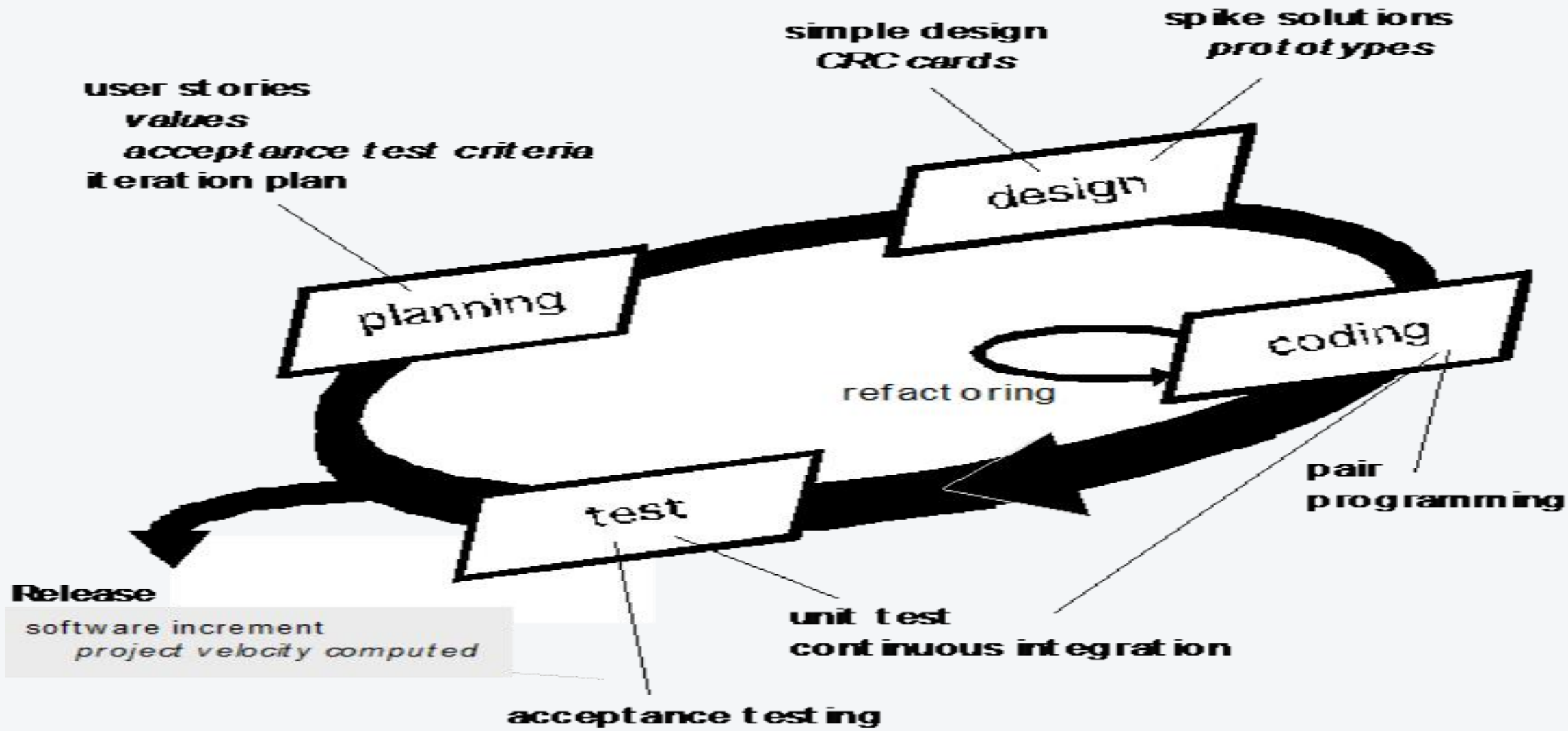
Great teams with average processes will outperform average teams with great processes. **People are the primary success factor.**

Key Traits of Agile Teams

- **Competence**
  Technical skills and expertise
- **Common Focus**
  Shared goals and vision
- **Collaboration**
  Working together effectively
- **Decision-Making Ability**
  Authority and autonomy to choose
- **Fuzzy Problem-Solving**
  Handle ambiguity and complexity
- **Mutual Trust & Respect**
  Foundation of team dynamics
- **Self-Organization**
  Autonomous team management

XP: Planning

The most widely used agile process, originally proposed by Kent Beck



Extreme Programming

XP: Planning

The most widely used agile process, originally proposed by Kent Beck

1. User Stories

Begins with the creation of **"user stories"** — short, simple descriptions of features told from the perspective of the user.

Example: "As a customer, I want to search for products so that I can find what I'm looking for quickly."

2. Cost Assessment

Agile team **assesses each story** and assigns a cost (typically in development hours or story points).

Estimation involves the whole team for accuracy and shared understanding.

3. Story Grouping

Stories are **grouped to form a deliverable increment** — a collection of stories that together provide meaningful functionality.

4. Delivery Commitment

A **commitment is made on delivery date** based on team velocity and story complexity. Dates are realistic.

5. Project Velocity

After the first increment, **"project velocity"** is used to help define subsequent delivery dates.

XP Planning

- Customer-driven stories
- Team-based estimation
- Data-driven planning

Key Point

XP planning is **adaptive and collaborative,** not rigid top-down planning.

Extreme Programming

XP: Design, Coding & Testing

XP Design

KIS Principle
Keep It Simple — avoid over-engineering

CRC Cards
Class-Responsibility-Collaborator cards

Spike Solutions
Design prototypes for difficult problems

Refactoring
Iterative refinement of internal design

XP Coding

Test-First Development

Construct unit test for a story **before coding** commences.

Pair Programming

Two developers work together at one workstation, improving quality and knowledge sharing.

XP Testing

Daily Unit Tests

All unit tests are **executed daily** to ensure continuous functionality.

Acceptance Tests

Defined by the customer to assess customer-visible functionality.

XP Philosophy

Extreme Programming takes **best practices to the extreme** — if testing is good, test all the time. If simplicity is good, keep it simple always.

Quality Outcomes

XP practices result in **higher quality code, fewer defects, better design, and improved team communication.**

Visual Overview

Extreme Programming

A comprehensive agile methodology

Core Practices

- ✓ User stories & incremental planning
- ✓ Test-driven development (TDD)
- ✓ Pair programming & collective ownership
- ✓ Continuous integration & refactoring

Core Values

- ✓ Communication & collaboration
- ✓ Simplicity & feedback
- ✓ Courage & respect
- ✓ Quality & sustainable pace

Team Structure

- ✓ On-site customer involvement
- ✓ Small, cohesive teams (5-15 people)
- ✓ Self-organizing & cross-functional
- ✓ 40-hour work week & sustainable pace

> "Extreme Programming is a discipline of software development based on values of simplicity, communication, feedback, courage, and respect."
>
> — Kent Beck

Extreme Programming

Key Points of XP

13 essential practices that define Extreme Programming

On-site Customer

Customer works with the team full-time to answer questions and provide feedback

User Stories

Short, simple descriptions of features from the user's perspective

Small Releases

Frequent, small increments of working software delivered to production

Pair Programming

Two developers work together at one workstation

Test-Driven Development (TDD)

Write tests before writing code

Collective Ownership

Anyone can change any code anywhere at any time

Continuous Integration

Integrate code multiple times per day

40-Hour Week

Sustainable pace with no overtime

The Planning Game

Collaborative planning between developers and customers

Simple Design

Minimal complexity, meets current needs

Refactoring

Continuously improve code quality

Metaphor

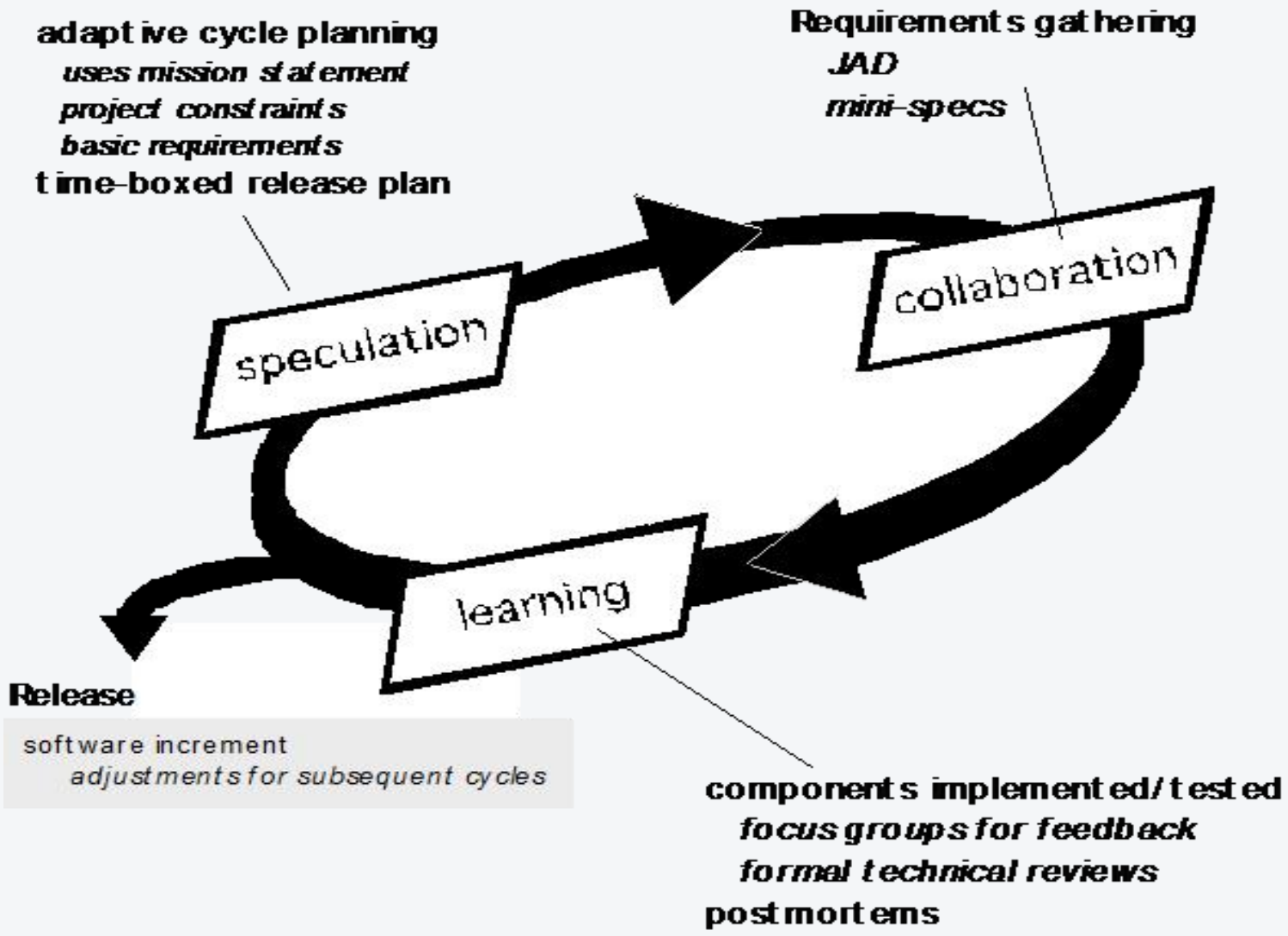
Shared story guiding development

Coding Standards

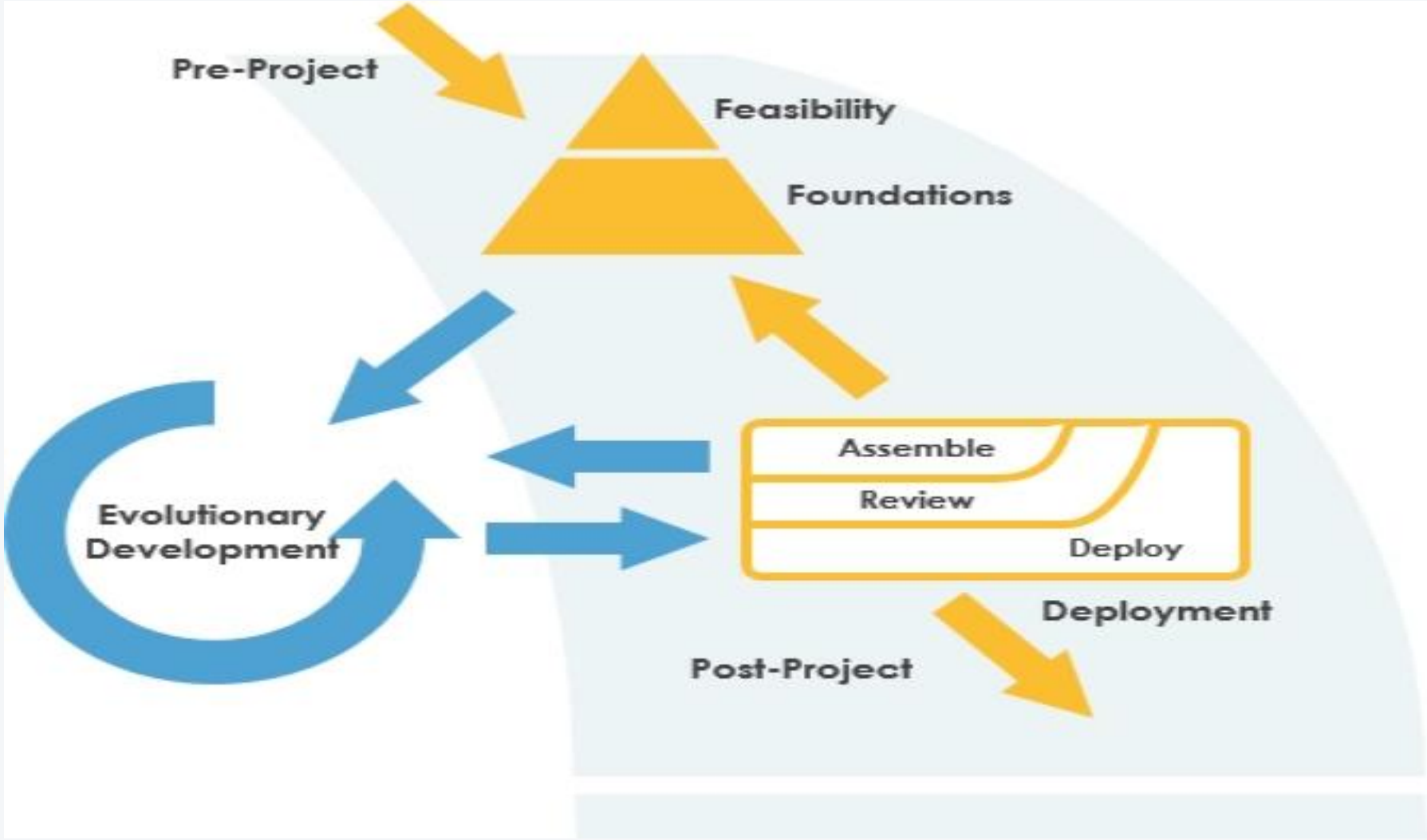
Consistent style and practices

ASD Framework

Originally proposed by Jim Highsmith

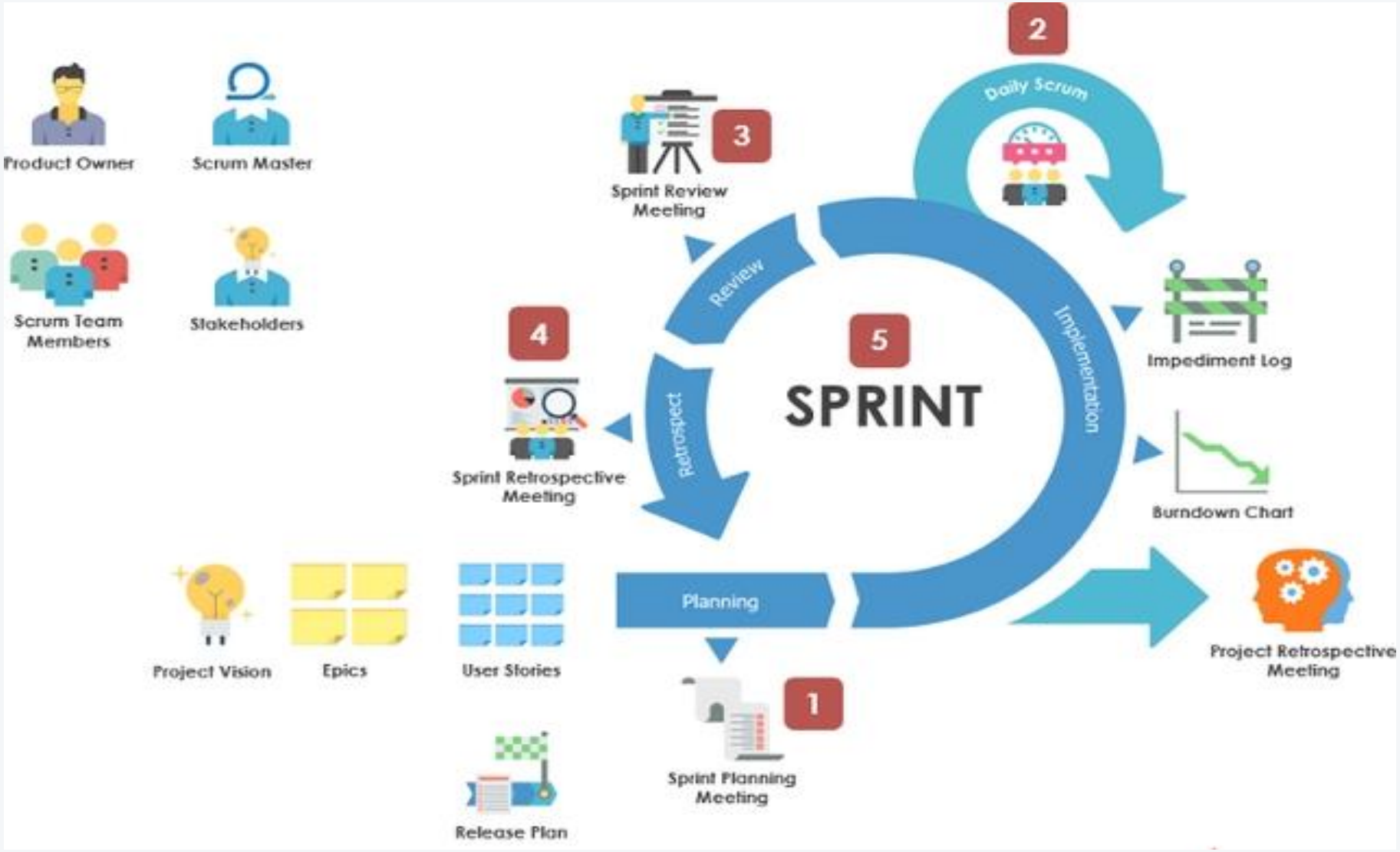


Dynamic Systems Development Method

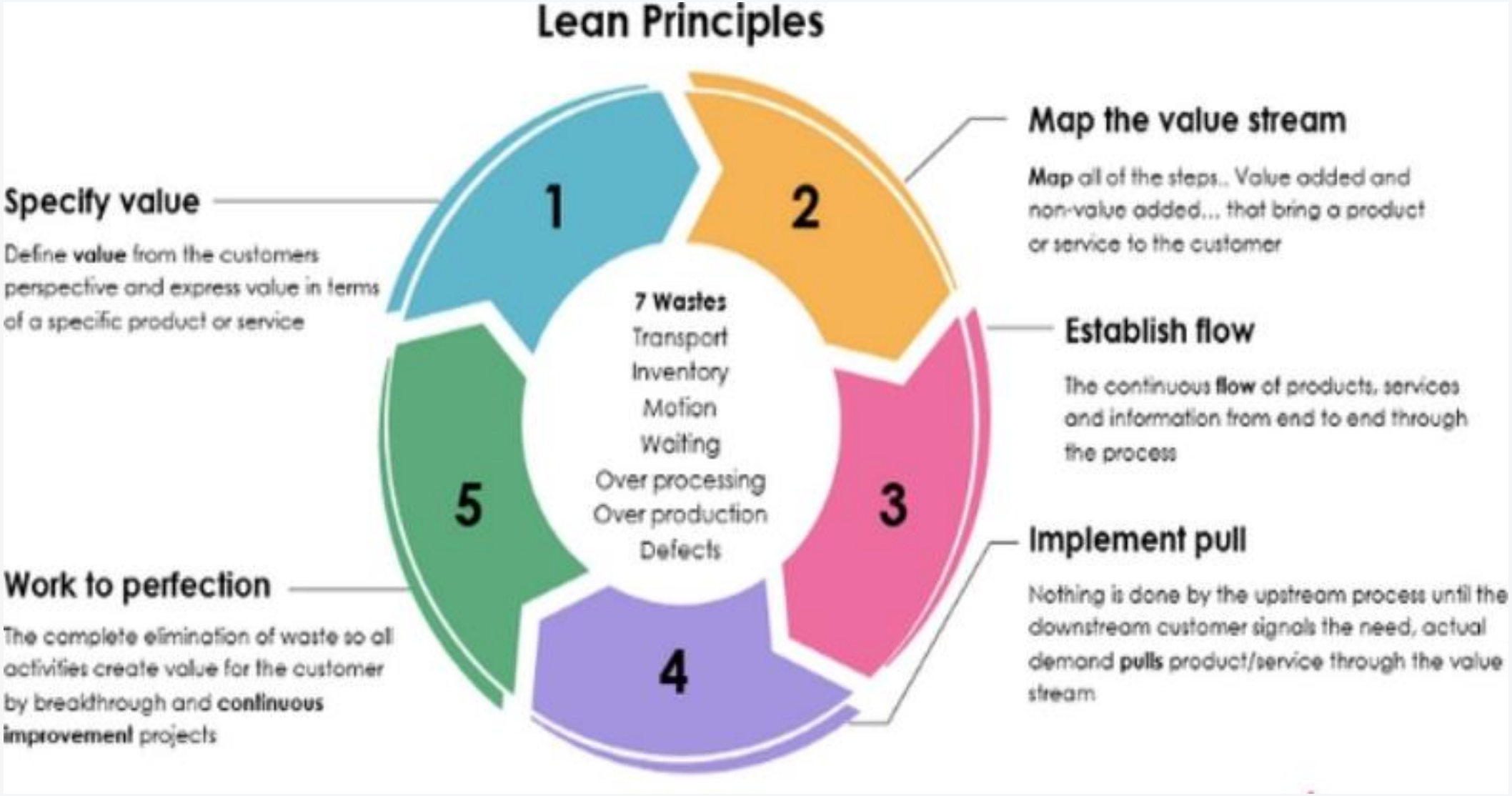


Scrum Framework

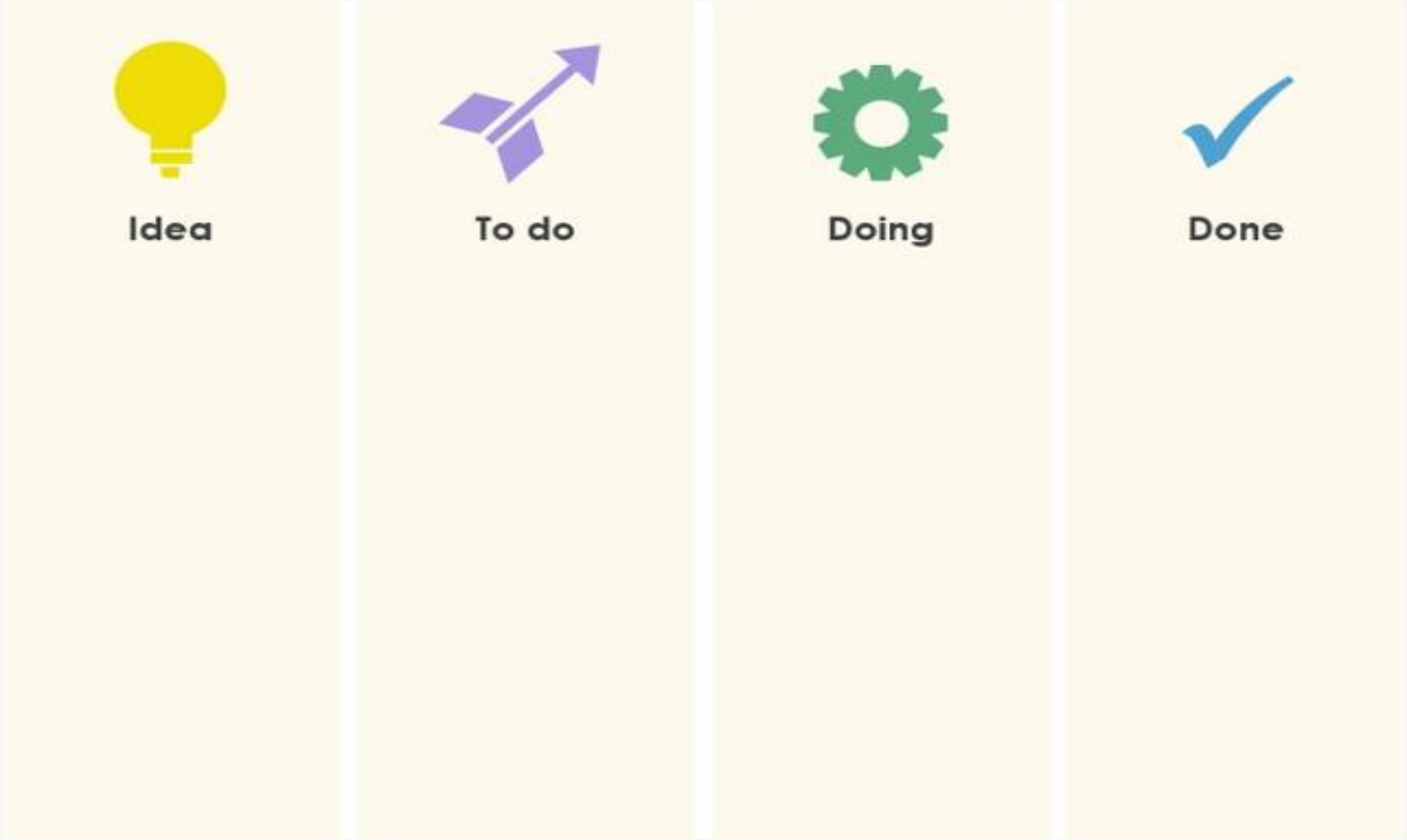
Scrum Methodology



Lean Software Development



KanBan Software Development



Chapter Summary

Four Key Themes of Agile

Self-Organization

Importance of self-organization — agile teams are autonomous and make their own decisions.

Self-organizing teams are more motivated, innovative, and responsive to change.

Communication & Collaboration

Communication and collaboration are central to agile success.

Face-to-face conversation is the most effective method of conveying information.

Change Recognition

Change recognition — agile embraces changing requirements as a natural part of development.

Agile processes harness change for the customer's competitive advantage.

Rapid Delivery

Rapid software delivery satisfies customers — early and continuous delivery of valuable software.

Working software is delivered frequently, from weeks to months.

Core Insight

These four themes work together to create a **people-centric, adaptive approach** to software development that delivers value quickly and responds to change effectively.

Chapter Summary

Extreme Programming Overview

XP is the most widely used agile process

Planning

- User stories
- Cost assessment
- Project velocity
- Release planning

Design

- Keep It Simple
- CRC cards
- Spike solutions
- Refactoring

Coding

- Test-first approach
- Pair programming
- Continuous integration
- Collective ownership

Testing

- Unit tests daily
- Acceptance tests
- Customer-defined
- Automated testing

XP Values

- **Communication**
- **Simplicity**
- **Feedback**
- **Courage**

Why XP Works

XP takes **best practices to the extreme** — if something is good, do it all the time.

- ✓ Emphasizes technical excellence
- ✓ Enables rapid feedback
- ✓ Builds quality in from the start
- ✓ Promotes collective ownership

Exercise

Questions (Part 1)

1. Agility is nothing more than the ability of a project team to respond rapidly to change. (B)
   A. True
   B. False

2. Which of the following is not necessary to apply agility to a software process? (A)
   A. Eliminate the use of project planning and testing
   B. Only essential work products are produced
   C. Process allows team to streamline tasks
   D. Uses incremental product delivery strategy

3. How do you create agile processes to manage unpredictability? (E)
   A. Requirements gathering must be conducted very carefully
   B. Risk analysis must be conducted before planning takes place
   C. Software increments must be delivered in short time periods
   D. Software processes must adapt to changes incrementally
   E. Both c and d

4. Which of the following traits need to exist among the members of an agile software team? (D)
   A. Competence
   B. Decision-making ability
   C. Mutual trust and respect
   D. All of the above.

Exercise

Questions (Part 2)

5 All agile process models conform to a greater or lesser degree to the principles stated in the "Manifesto for Agile Software Development". Answer: A

A. True

B. False

6 What are the four framework activities found in the Extreme Programming (XP) process model? Answer: D

A. analysis, design, coding, testing

B. planning, analysis, design, coding

C. planning, analysis, coding, testing

D. planning, design, coding, testing

7 What are the three framework activities for the Adaptive Software Development (ASD) process model? Answer: D

A. analysis, design, coding

B. feasibility study, functional model iteration, implementation

C. requirements gathering, adaptive cycle planning, iterative development

D. speculation, collaboration, learning

8 Which is not one of the key questions that is answered by each team member at each daily Scrum meeting? Answer: C

A. What did you do since the last meeting?

B. What obstacles are you encountering?

C. What is the cause of the problems you are encountering?

D. What do you plan to accomplish at the next team meeting?